

Laboratory Exercise No. 3: Building and Training a Neural Network for Classification Using Python

Library: scikit-learn

Algorithm: Isolation Forest

Estimated Time: 2 hours

1. Learning Objectives

At the end of this laboratory activity, students should be able to:

- ✓ Create a simple dataset for classification.
- ✓ Prepare training and testing data.
- ✓ Create a neural network using Python.
- ✓ Train the neural network using training data.
- ✓ Generate predictions using the trained model.
- ✓ Evaluate the model's performance.
- ✓ Explain the relationship between the Python program and the neural network training process.

2. Scenario

Imagine that a school wants to predict whether a student is likely to **PASS or FAIL** based on:

- ✓ Study Hours
- ✓ Attendance

For example:

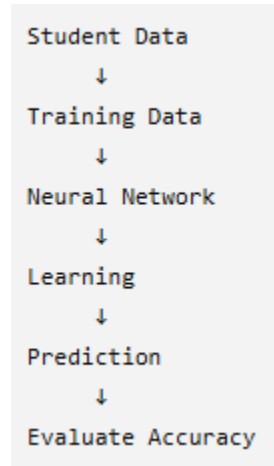
Study Hours	Attendance (%)	Result
1	55	Fail
2	60	Fail
2	65	Fail
3	70	Pass
4	75	Pass
5	80	Pass
6	85	Pass
7	90	Pass

The neural network will learn the relationship between these inputs and the result.

3. What to Build?

Train a computer to predict whether a student will pass or fail based on study hours and attendance.

The overall process is:



4. Required Software

Students need:

- ✓ Python 3
- ✓ Visual Studio Code / PyCharm / IDLE / NetBeans with Python support
- ✓ scikit-learn
- ✓ pandas
- ✓ matplotlib

5. Create a Python Program

Create a file:

`neural_network.py`

5.1. Step One: Import the Libraries

Python

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report
```

You don't need to memorize these imports.

"pandas handles the dataset. scikit-learn provides the neural network and evaluation tools. matplotlib allows us to visualize the data."

5.2. Step Two: Create Datasets

```
data = {
    "study_hours": [1, 2, 2, 3, 4, 5, 6, 7, 1, 3, 4, 6],
    "attendance": [55, 60, 65, 70, 75, 80, 85, 90, 50, 68, 78, 88],
    "result": [0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1]
}

df = pd.DataFrame(data)

print(df)
```

Where:

```
0 = Fail
1 = Pass
```

So:

```
Study Hours + Attendance → Result
```

For example:

```
7 hours + 90% attendance → 1 → Pass
```

5.3. Step Three: Separate Inputs and Outputs

```
X = df[["study_hours", "attendance"]]  
y = df["result"]
```

X contains input features:

```
Study Hours  
Attendance
```

Y contains the target/output:

```
Pass / Fail
```

This connects directly to the neural-network architecture.

```
INPUT  
↓  
Study Hours  
Attendance  
↓  
Neural Network  
↓  
OUTPUT  
↓  
Pass / Fail
```

5.4. Step Four: Split the Dataset

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.25,  
    random_state=42,  
    stratify=y  
)
```

We don't want to train and test the model using the same data.

We divide the data into:

```
75% → Training  
25% → Testing
```

The training data teaches the model. The testing data checks whether the model can make predictions on data it did not use for training.

5.5. Step 5 — Create the Neural Network

Create the actual neural network:

```
model = MLPClassifier(  
    hidden_layer_sizes=(5,),  
    activation="relu",  
    solver="adam",  
    max_iter=1000,  
    random_state=42  
)
```

`MLPClassifier`

MLP means MULTI-LAYER PERCEPTION. It is a feedforward neural network classifier.

`hidden_layer_sizes=(5,)`

This means

```
Input Layer  
  ↓  
5 Neurons  
  ↓  
Output Layer
```

`activation="relu"`

We use the ReLU activation function, which will be introduced in your lecture.

`solver="adam"`

Adam is an optimization method used during training. In the lecture, it identifies Adam as one of the gradient-descent optimization variants.

5.6. Step Six – Train the Neural Network.

```
model.fit(X_train, y_train)
```

This is where the neural network learns. This one line triggers the training process.

Conceptually, the computer is performing something similar to:

```
Forward Pass
↓
Calculate Loss
↓
Backpropagation
↓
Update Weights
↓
Repeat
```

This is to be identified in our lecture. This step is the fundamental training cycle.

5.7. Step 7 — Make Predictions

```
y_pred = model.predict(X_test)

print("Actual:")
print(y_test.values)

print("Predicted:")
print(y_pred)
```

should see something similar to:

```
Actual:
[1 0 1]

Predicted:
[1 0 1]
```

The exact output may vary depending on the dataset and training process.

5.8. Step 8 — Calculate Accuracy

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
```

For example:

```
Accuracy: 1.0
Accuracy Percentage: 100.0 %
```

The accuracy describes how well the model performed on this particular test set. Because our dataset is very small, we should not assume that the model will perform equally well on a much larger real-world dataset.

This is a good opportunity to introduce **generalization and overfitting**, which our lecture identifies as challenges in deep learning.

5.9. Step 9 — Classification Report

Add:

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

You will see metrics such as:

```
precision
recall
f1-score
support
```

Accuracy gives us one overall measure, but the classification report gives us more detailed information about the predictions.

5.10. Step 10 — Test a New Student

Ask the model to predict a completely new example:

```
new_student = [[5, 82]]

prediction = model.predict(new_student)

if prediction[0] == 1:
    print("Prediction: PASS")
else:
    print("Prediction: FAIL")
```

The model receives:

```
Study Hours = 5
Attendance = 82%
```

and produces:

```
Prediction: PASS
```

The exact result can depend on the trained model and dataset.

5.11. Step 11 — Test Multiple Students

```
new_students = [
    [1, 50],
    [2, 60],
    [4, 75],
    [5, 82],
    [7, 95]
]
```



```

predictions = model.predict(new_students)

for i, prediction in enumerate(predictions):
    if prediction == 1:
        result = "PASS"
    else:
        result = "FAIL"

    print(
        "Student", i + 1,
        "Study Hours:", new_students[i][0],
        "Attendance:", new_students[i][1],
        "Prediction:", result
    )

```

You should see the relationship between:

```

Study Hours
+
Attendance
↓
Pass / Fail

```

5.12. Complete Python Program

5.12.1. Task 1 — Run the Program

Run the complete program successfully.

Take a screenshot showing:

5.12.1.1. Dataset

5.12.1.2. Actual values

5.12.1.3. Predicted values

5.12.1.4. Accuracy

5.12.1.5. Classification report

5.12.2. Task 2 — Change the Dataset

Add at least **5 additional students**.

For example:

```
study_hours = 8  
attendance = 92  
result = 1
```

You should add their own records. Run the program again.

Did the accuracy change? Why?

5.12.3. Task 3 — Change the Neural Network

Change:

```
hidden_layer_sizes=(5,)
```

To

```
hidden_layer_sizes=(10,)
```

Run the program again.

Then try:

```
hidden_layer_sizes=(10, 5)
```

What changed when you added another hidden layer?

This directly reinforces the lecture's concept of **deep networks and hierarchical representations**.

5.12.4. Task 4 — Change the Activation Function

Change:

```
activation="relu"
```

To

```
activation="logistic"
```

Run the program again.

Did the model's result change?

5.12.5. Task 5 — Test Your Own Student

Create a prediction for a hypothetical student using:

```
Study Hours = _____  
Attendance = _____
```

Record the model's prediction.

6. Guide Questions.

6.1. You should answer these after completing the activity.

6.1.1. What are the **input features** in this activity?

6.1.2. What is the **target/output**?

6.1.3. What does `MLPClassifier` represent?

6.1.4. What is the purpose of the hidden layer?

6.1.5. What does the ReLU activation function do?

6.1.6. What happens during `model.fit()`?

6.1.7. How does the neural network learn from errors?

6.1.8. What is the purpose of the test dataset?

6.1.9. What does accuracy measure?

6.1.10. What happened when you increased the number of neurons?

6.1.11. What happened when you added another hidden layer?

6.1.12. Why does adding more layers not automatically guarantee better results?

6.2. Expected Learning Connection

At the end of the laboratory, students should be able to connect:

Lecture Concept	Python Implementation
Input Layer	X
Output/Target	y
Neural Network	MLPClassifier
Hidden Layer	hidden_layer_sizes
Activation Function	activation="relu"
Training	model.fit()
Prediction	model.predict()
Loss/Optimization	Internal training process
Backpropagation	Internal training process
Evaluation	accuracy_score()
Classification	Pass/Fail

7. Submission Requirements.

7.1. Students should submit:

7.1.1. Python source code

```
neural_network.py
```

7.1.2. Screenshot of successful execution

7.1.2.1. Dataset

7.1.2.2. Prediction

7.1.2.3. Accuracy

7.1.2.4. Classification report

7.1.3. Short Laboratory Report

7.1.3.1. Objective

7.1.3.2. Procedure

7.1.3.3. Results

7.1.3.4. Answers to Guide Questions

7.1.3.5. Conclusion

For the conclusion, explain:

7.1.3.5.1. How does the Python program demonstrate the neural-network learning process discussed in the lecture?